UCLA Samueli School of Engineering

CS 35L Software Construction

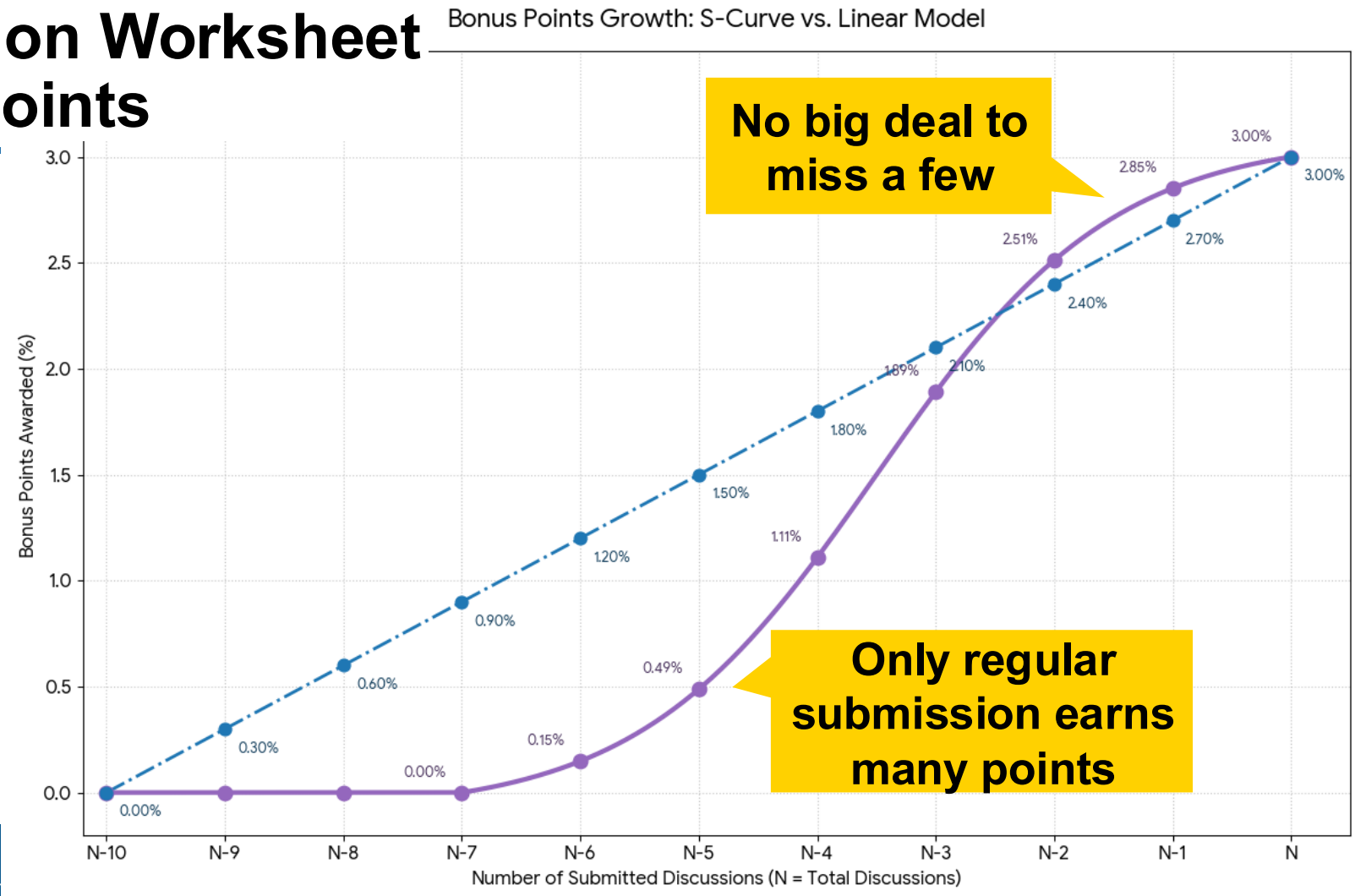
L2 – Files & Shell Scripting

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department

Course Logistics

- **Homework 1 will be released today**. Due in **one week**
  - Writing User Stories & Justifying INVEST
  - Creating a shell script **on SEASNet**
    - If you have not requested an account yet, do so today
- Discussions on Friday
  - Provide **Hands-on Experience**
  - Give you a **head start on the Homework**

Discussion Worksheet
Bonus Points



UCLA Samueli School of Engineering

Different Course Resources are Designed for Different Students. Use them If these Apply to You

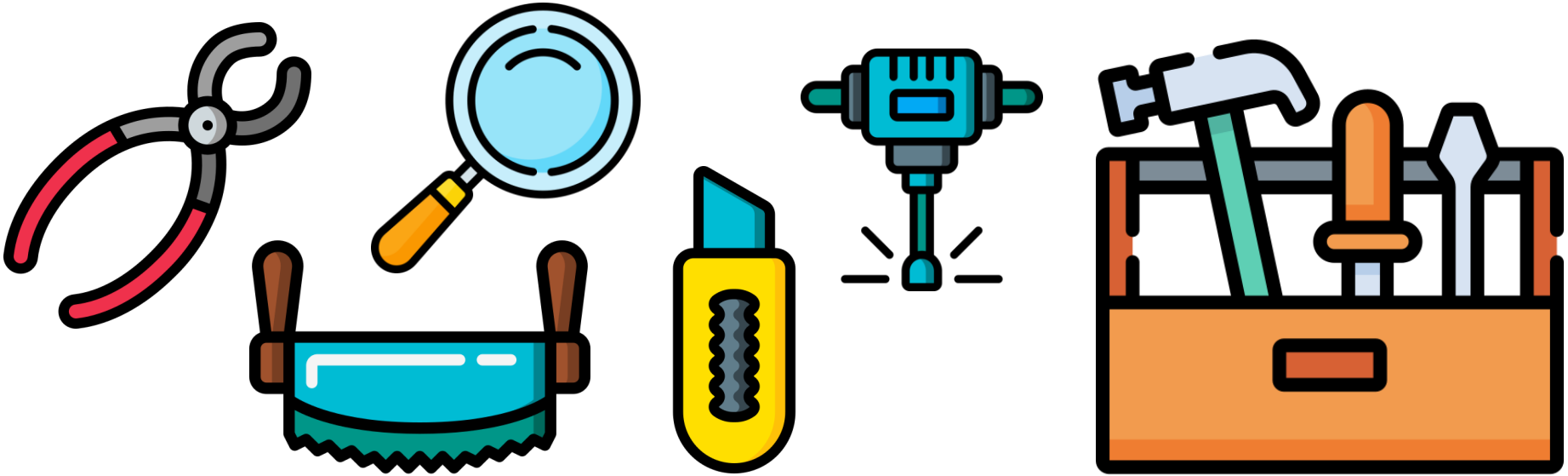
- If a topic is **unclear to you after the lecture**
  - Read the **SE Book** page on the topic
  - Ask on **Piazza** if you believe others might have the same question
  - Come to TA/LA/Prof **office hours** We're eager to support you
- If you want **additional practice**
  - Go to **discussions**
  - Talk to the **digital tutors**. They will give you practice problems & feedback
- If you already **know everything** and you're **bored** by the class
  - Come to Prof office hours . Read SE Book pages and further readings

How to use Course Resources?

- **Always** walk through the tutorials we link!
  - The **discussions** will give you a head start
  - Then you should finish them at **home**

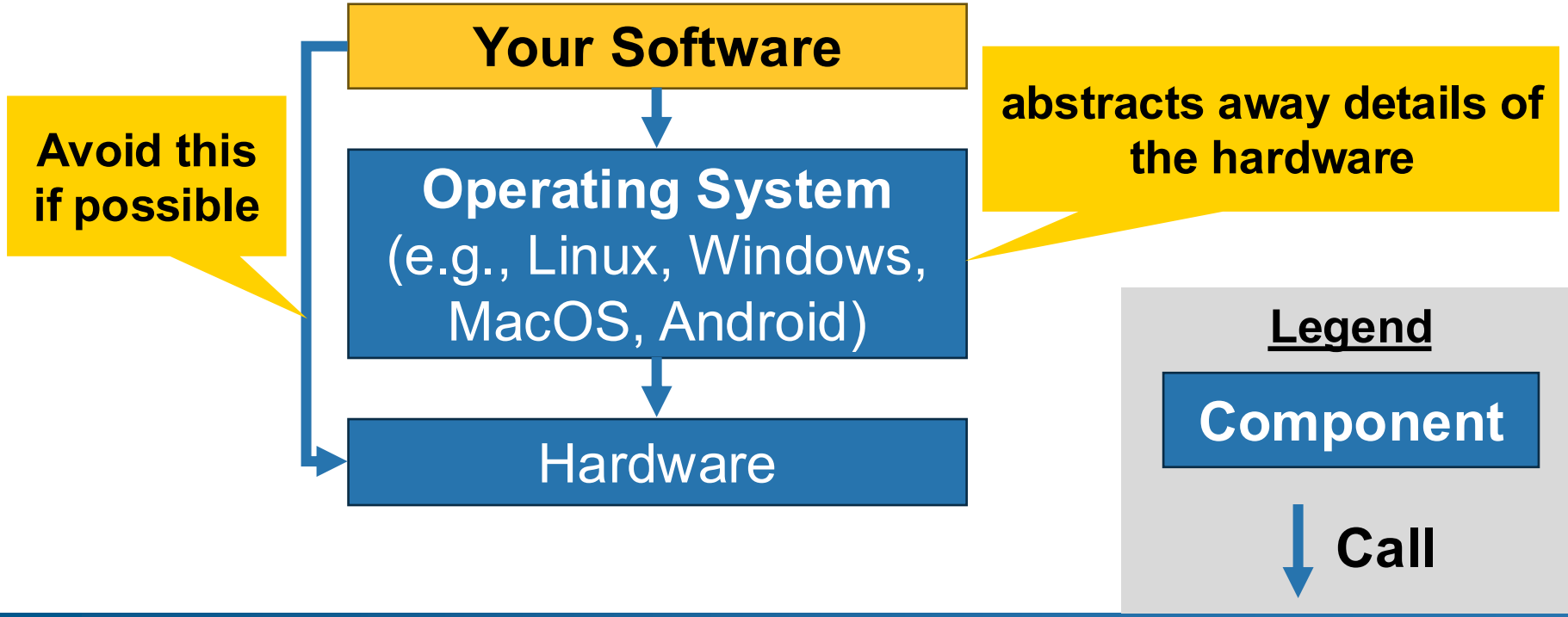
- If you are in this class to **learn** and/or get a **good grade** (hopefully everyone)
  - **Regularly Practice**  the quiz & flashcard questions in the **SE Gym**💪

A Good Software Engineer has a Great Understanding of Existing Tools



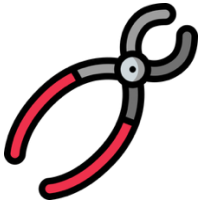
Today's lectures teaches you how to use common UNIX tools

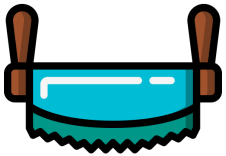
Operating Systems offer Useful Functionality



UNIX offers you Many Useful Tools

- UNIX is a family of multitasking, multi-user computer operating systems.
- Arguably the most influential operating system of all time, serving as the foundation or inspiration for systems like Linux, macOS, and Android
- Unix design philosophy:
  - "Write programs that do one thing and do it well.“
  - “Write programs to work together.”
  - “Write programs to handle text streams, because that is a universal interface.”





UNIX Commands for File Handling

- `cd <dir>` (**c**hange **d**irectory)
- `ls` (**ls**t files)
- `mkdir <dir>` (**m**a**k**e **dir**ectory)
- `cp <source> <destination>`: (**c**o**p**y file)
- `mv <source> <destination>`: (**m**o**v**e/rename file)
- `rm <file>`: delete file
- `less <file>`: view file contents one screen at a time
- `cat <file1> <file2>`: Con**cat**enate files and print their content to standard output. Often used to quickly view a small file

Please familiarize yourself with these commands

UNIX Commands for Text Processing

- `sed` (**stream editor**) used for filtering and **transforming text**, most commonly for search-and-replace operations
- `grep` (**global regular expressions**) find and replace
- `head`/`tail` `<file>`: prints the first / last lines of a file
- `wc` `<file>`: **w**ord **c**ount **(counts characters, words, lines)**
- `sort`: sorts lines in a file
- `cut`: Extracts characters / fields / bytes at specific positions / delimiters / offsets

Please familiarize yourself with these commands

Other Frequently Used UNIX Commands

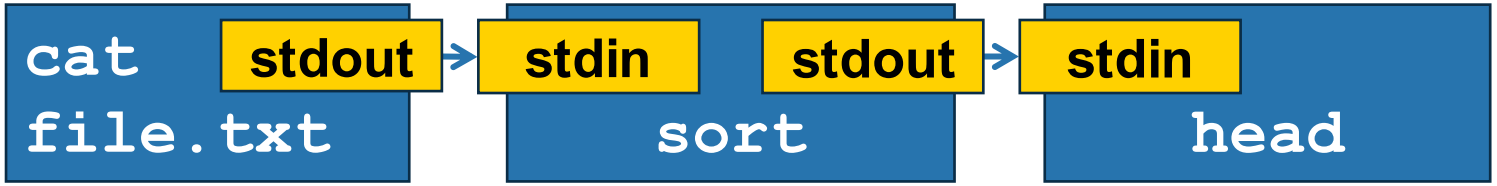
- `ssh <remote>`: **s**ecure **sh**ell.
  Used connect to a remote computer
- `htop`: shows system monitoring tools
- `man <command>`: **man**ual for a command
- `pwd` (**p**rint **w**orking **d**irectory) checks your current path
- `chmod`: (**ch**ange **mode**). Sets the permissions for a file
  - Files have three types of permission:
    - READ (r)
    - WRITE (w)
    - EXECUTE (x)

Please familiarize yourself with these commands

“Write programs to handle text streams, because that is a universal interface.” – UNIX Philosophy

- To handle inputs and output, UNIX programs have:
  - a **standard input** stream port **(stdin)**
  - a **standard output** stream port **(stdout)**
  - a **standard error** stream port **(stderr)**

  } **These are text streams**
- Additional inputs can be added via command line arguments
- **Composition**: By connecting ports of multiple commands, you can create a larger program




```
cat file.txt | sort | head
```

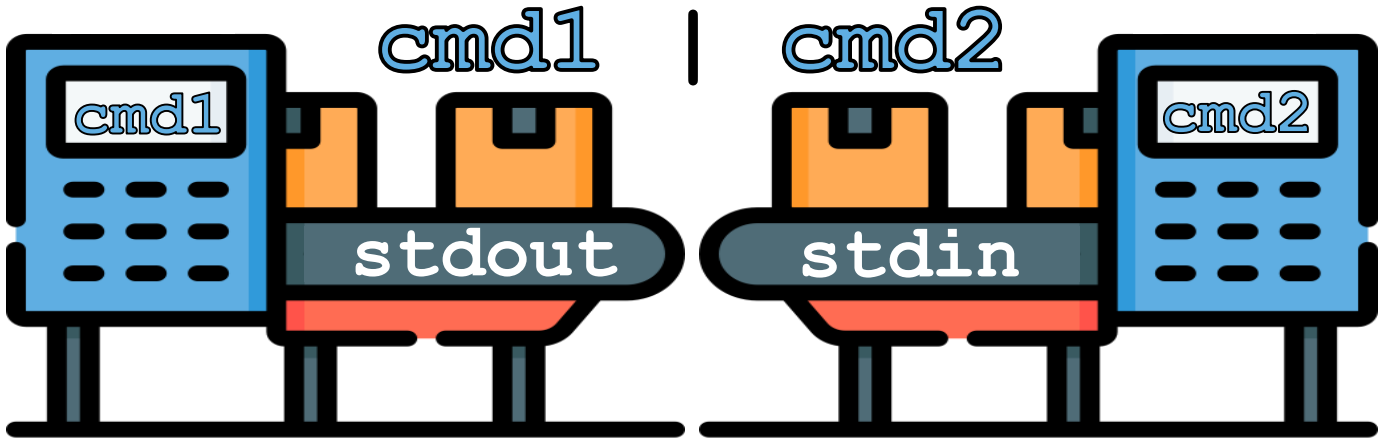
Legend

Program

Port

Pipe

Streams are sequences of characters / bytes



Streams are like conveyor belts, not like trucks

Streams **transport portions of data individually** rather than all at once

\+ Allows for **parallel processing**

\- Can result in **indefinite waiting** for stream to close

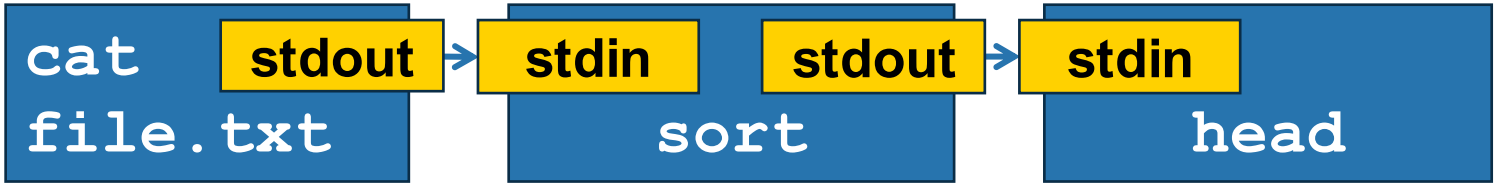
I/O (Input & Output) Streams

- Reading from a file: **`cmd < file`**
  - Reads the file and forwards it to the standard input (**`stdin`**) of the cmd
- Writing to file: **`cmd > file`** or **`cmd 1> file`**
  - Writes the content of the standard output (**`stdout`**) stream to the file
- Logging errors to a file: **`cmd 2> file`**
  - Writes the content of the standard error (**`stderr`**) stream to the file
- Forwarding **`stdout`** of **`cmd1`** to **`stdin`** of **`cmd2`**: **`cmd1 | cmd2`**
- Redirecting stderr to stout: **`cmd 2 >& 1`**
- Closing the stdin stream: **`cmd <&-`**

“Write programs to handle text streams, because that is a universal interface.” – UNIX Philosophy

Are text streams *really* a universal interface?

Discuss with your neighbor(s)




```
cat file.txt | sort | head
```

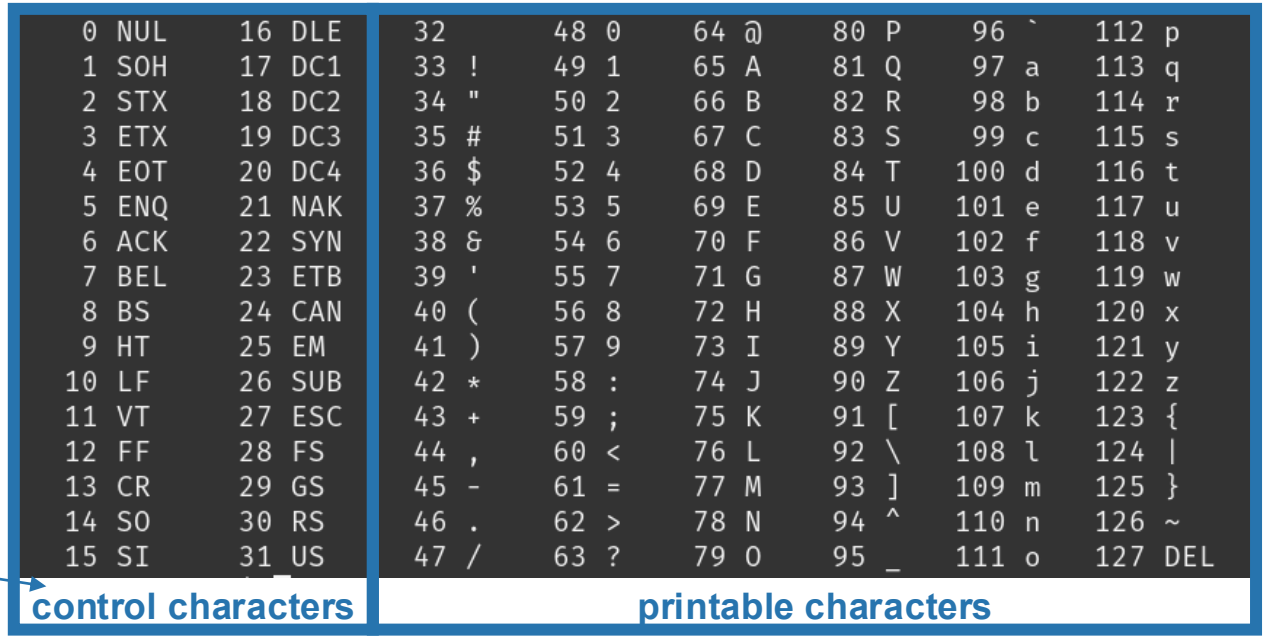
Case Study: Nametags

What happened here?

ASCII (American Standard Code for Information Interchange)

- ASCII encodes each character as a **value from 0 to 127** – storable as a **seven-bit integer**

non-printing codes primarily designed to control peripheral devices (like printers or terminals) or to provide meta-information about a data stream



Read more here: https://www.ascii-code.com/

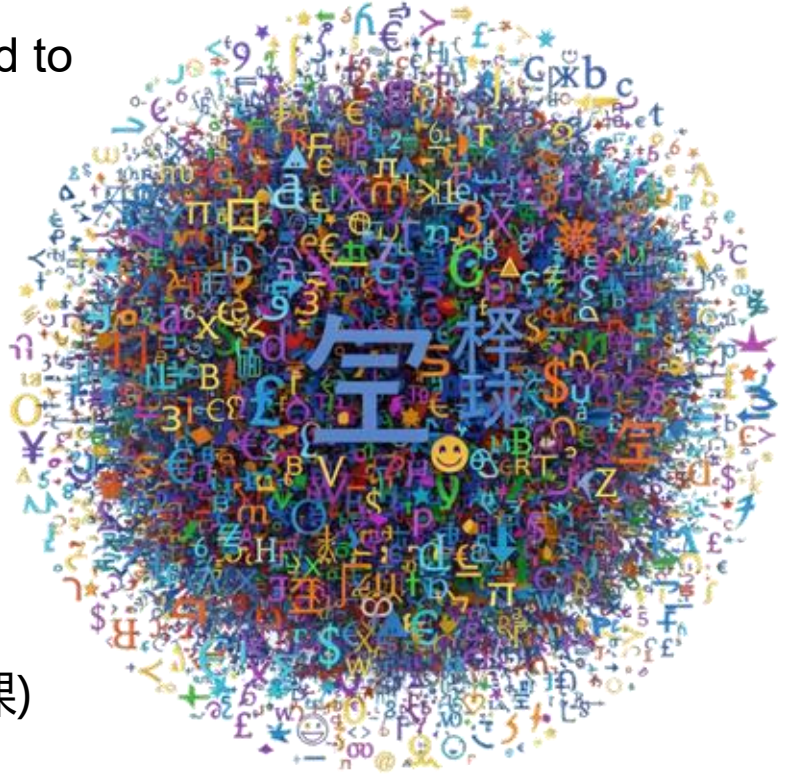
There are Several Different Variations of ASCII Extensions to 8-bit

- Windows Code Pages aka. ANSI Code Pages (e.g., Windows-1252)
  - **Standard for Western European languages in Microsoft Windows**
  - Includes ü, é, ê, è, …
- ISO 8859 Series (e.g., ISO 8859-1 for Latin-1, ISO 8859-2 for Latin-2)
  - **Offers different extension parts for several languages** (e.g., Greek, Arabic, Hebrew, Thai, …)

Read more here: https://en.wikipedia.org/wiki/ISO/IEC_8859

UTF-8 (Unicode Transformation Format – 8-bit.)

- Unicode (aka The Unicode Standard) is designed to support **all of the world's writing systems** that can be digitized
- UTF-8 supports all 1,112,064 valid Unicode characters using **variable-width encoding** of one to four one-byte (8-bit) code units
- Emojis (e.g., 👦🎓 👦🏫)
- Korean Alphabet (e.g.,한글)
- Chinese Characters (e.g., 你应该好好学习这门课)



Read more here: https://home.unicode.org/

What *exactly* happened here?

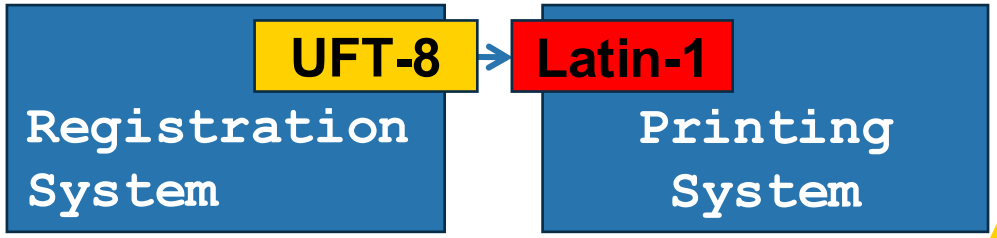
ü in UTF-8 stored as bytes: (C3 BC)

In Latin-1 / Windows-1252:

C3 → Ã

BC → ¼

Result: Dürschmid → DÃ¼rschmid



Why did all other characters print correctly?

Where does the ? come from?

The UNIX Shell lets you use UNIX commands

- A **command-line interpreter** that provides a **command line user interface**
- Typically, users interact with the UNIX shell using a **terminal application**
- **Shell is the 6th most common tool used by practitioners**
  (after JavaScript, HTML/CSS, Python, SQL, and TypeScript,
  most of which you will see in this course!)

Shell Scripts

Hello.sh

Tells the system to use the sh shell

```
#!/bin/sh

$1 reads the first command line argument

echo "Hello $1! Let's get scripting."
```

Terminal

Gives users the "read" and the "execute" permissions

command line argument

```
$chmod +x hello.sh
$./hello.sh students
Hello students! Let's get scripting
```

Shell Scripts Support Variables

HelloVar.sh

```
#!/bin/sh
echo "What is your name?"

read USER_NAME

echo "Welcome, $USER_NAME! Let's get scripting."
```

`read` stores the user input in the `USER_NAME` variable

`$USER_NAME` gets the value of the `USER_NAME` variable

Terminal

```
$ ./hellovar.sh
What is your name?
Tobias
Welcome, Tobias! Let's get scripting.
```

Shell Scripts Support For Loops

for.sh

```
#!/bin/sh
for i in 1 2 3 4
do
  echo "Looping ... number $i"
done
```

Terminal

```
$ ./for.sh
Looping ... number 1
Looping ... number 2
Looping ... number 3
Looping ... number 4
```

Shell Scripts Support If Then Else

if.sh

```
#!/bin/sh
m=1
n=2

if [ $n -eq $m ]
then
        echo "Both variables are the same"
else
        echo "Both variables are different"
fi
```

Shell Scripts Support Wildecards

clean.sh

. Start the search in the current directory.
-type f : Only match regular files (not directories or links).
-iname "*.log": Match files ending in .log, ignoring case
-delete : Permanently deletes the files found

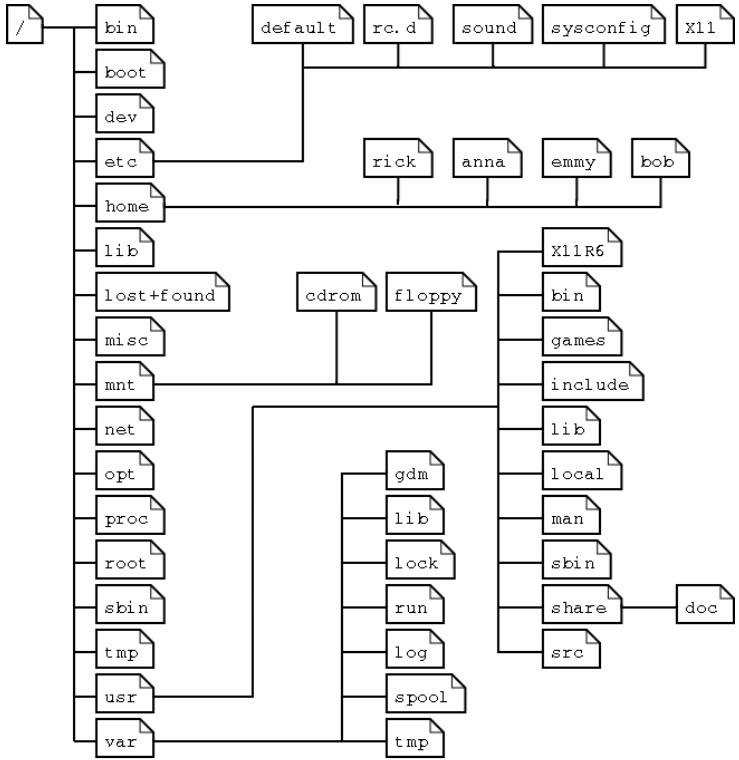
```
#!/bin/sh

find . -type f -iname "*.log" -delete
```

$? holds the exit status (or return code) of the last executed command.
0 means no errors, other numbers indicate the error code

```
if [ $? -eq 0 ]; then
    echo "Cleanup complete. All *.log files have been removed."
else
    echo "An error occurred during cleanup"
fi
```

Common Directories

- **`/bin:`** Common programs (short for binary)
- **`/dev:`** Peripheral hardware
- **`/etc:`** Configurations
- **`/home:`** Home directories of users
- **`/lib:`** Library files
- **`/mnt:`** Mount point for external FS
- **`/opt:`** Extra software
- **`/root:`** Home for root
- **`/tmp:`** Temporary files
- **`/usr:`** Files for user-related programs
- **`/var:`** Variable and temporary files



Paths

- Absolute path (starts from the root directory):
  - e.g., /usr/local/
- Relative path (to access files relative to your current working directoru):
  - ./ = current directory
  - ../ = parent directory
  - ~/ = home directory
- Use **`pwd`** (**p**rint **w**orking **d**irectory) to check your current path

The PATH Environment Variable

- The **PATH** environment variable contains an ordered list of directories that the operating system (like Windows, Linux, or macOS) searches for executable programs when a command is entered in a command line interface (CLI).
- If the CLI cannot find the program / file in your working directory, it will then search through the directories listed in the PATH.
- Add a new directory to the path via:
  ```
  export PATH="\$PATH:/path/to/your/new/directory"
  ```
  - This change will be temporary for your current session.
  - To make the change permanent change add it to the `~/.bashrc` and/or `~/.zshrc` files

Regular Expressions (RegEx)

Useful for:

- **Data Validation**: Checking if a user-entered email address or phone number is in the correct format.
- **Text Processing**: Extracting all URLs, dates, or specific identifiers from a large document.
- **Search and Replace**: Finding all instances of a misspelled word, regardless of capitalization, and replacing it.
- **Log File Analysis**: Searching through system logs for errors or specific events.

Literal Characters (The Simple Stuff🤷)

Literals are characters that match themselves exactly

Pattern	Matches
a	“a”
123	“123"
HeLLo	“HeLLo”

Character Classes (Selecting one of many🎯)

Character classes define a set of possible characters to be matched

Pattern	Description	Example	Matches
[ ]	**Character Set:** Matches **any one** of the characters inside the brackets	[aeiou]	'a', 'e', 'i', 'o', or 'u'
[ ]	**Character Set:** Matches **any one** of the characters inside the brackets	[0-4]	'0', '1', '2', '3', '4'
[^]	**Negated Set:** Matches **any character NOT** inside the brackets.	[^0-9]	Any character that is NOT a digit.

Meta Characters (To make your life easier ☺)

Meta characters are short cuts for commonly used character classes

Pattern	Description
.	Matches **any single character** (except newline)
\d	Matches any digit (same as [0-9])
\D	Matches any non-digit (same as [^0-9])
\w	Any word character (same as [a-zA-Z0-9_]
\s	Any whitespace character (same as [\t\r\n\f])

Quantifiers (How many times? )

Quantifiers describe how many occurrences of the preceding element should be matched

Pattern	Description	Example	Matches
X*	Matches X **zero or more** times	\s*	Any white spaces
X+	Matches X **one or more** times	\d+	Any number
X?	Matches X **zero or one** times	colou?r	"color", "colour"
X{n}	Matches X **exactly n** times	\d{4}	Numbers with 4 digits
X{n,}	Matches X **n or more** times	\w{5,}	Words with 5+ characters
X{n,m}	Matches X **n to m** times	a{1,3}	"a", "aa", or "aaa"

Group Constructs (The Advanced Stuff🔥)

Group constructs allow you to **combine multiple characters or sub-patterns into a single unit**.

Pattern	Description	Example	Matches
(…)	Simple group	(ab)*	"ab", "abab", "ababab", …
(a | b)	Matches either a or b	(911|9-1-1)	“911”, “9-1-1”
(?<name>X)	Gives group matching X the name ‘name’	(?<num>[0-9]) (?<char>[a-Z])	“9a”, “8z”

Helps with parsing files

num char

Anchors 

Regular expressions can match sub-strings in a longer sequence. Anchors constrain matches based on their location inside the input sequence

Pattern	Description
^	Start of string Does not allow any other characters to come before the ^
$	End of string

In-Class Exercise

Write a regular expression that can be used to validate a password that must be at least 8 characters long and can contain only letters and digits

^[a-zA-Z0-9]{8, }$

RegEx Reference: https://tobiasduerschmid.github.io/SEBook/tools/regex.html

In-Class Exercise

Write a regular expression that can be used to validate an email address

Dots outside of character classes need to be escaped

```
^[a-zA-Z0-9.-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

- `[a-zA-Z0-9.-]+` — username can include letters, digits, dots, and dashes. (e.g, cs-35L)
- `[a-zA-Z0-9.-]+` — domain can include letters, digits, dots, and dashes (e.g., ucla2025)
- `[a-zA-Z]{2,}` — top level domain can include letters

RegEx Reference: https://tobiasduerschmid.github.io/SEBook/tools/regex.html

Please Fill out your Exit Tickets on Bruin Learn!

Question 1 — 1 pts

Please briefly summarize the **three key insights** you have learned today. Do not copy phrases from the lecture slides (and don't look back, answer purely from memory).

Question 2 — 1 pts

Which shell command goes into the folder "homework" and then lists all files within this folder? Write it as a one-line command!

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slide use images from Flaticon.com (Creators: Freepik, Smashicons) and generated with Gemini

Additional Resources

- https://tobiasduerschmid.github.io/SEBook/tools/shell
- https://tobiasduerschmid.github.io/SEBook/tools/shell-tutorial
- https://tobiasduerschmid.github.io/SEBook/tools/regex
- https://tobiasduerschmid.github.io/SEBook/tools/regex-tutorial
- https://tobiasduerschmid.github.io/SEBook/tools/regex-tutorial-advanced

Summary

- The operating system abstract away hardware details
- UNIX commands offer you a large variety of useful tools
- The UNIX shell lets you write small programs that use UNIX commands
- Make sure you use correct character encoding when reading / writing files
- In most cases, UTF-8 is the most appropriate encoding to use
- RegEx is a useful tool for text processing

Credits: These slide use images from Flaticon.com (Creators: Freepik, Smashicons)